

Reparto de Tareas

Proyecto Final

FastAPI · Streamlit · PostgreSQL · SQLAlchemy · Docker · GitHub Actions · Hexagonal

Equipo

Ayelén

Vaishali

Pamela

Giuliana

Paula

■ Cronograma

Jueves 30 abril → Domingo 4 mayo

Día	Qué debe estar listo
Jue 30	Ayelén: domain + interfaces + ORM. Vaishali: DTOs definidos.
Vie 1	Ayelén: infra + auth API. Vaishali: use cases + patrones. Paula: endpoints de solicitudes.
Sáb 2	Paula: endpoints notif/audit + UML. Pamela: login + listado + formulario. Giuliana: acciones + notificaciones.
Dom 4	Todo integrado. Tests. Docker Compose. Deploy dev/prod. README final. Demo preparada.

■■ El frontend (Pamela y Giuliana) no puede avanzar hasta que Ayelén tenga auth y Paula tenga los endpoints de solicitudes.

■■■ Ayelén

Backend Core + Infraestructura + DevOps

■■ Todos dependen de su trabajo. Es la primera en ejecutarse.

Domain & Entidades

Cada entidad debe ser clase Python pura — sin imports de SQLAlchemy ni FastAPI.

- Clases en domain/entities/: User, Role, Request, Approval, AuditLog, Notification
- Enums en domain/enums.py: RequestStatus (SUBMITTED, APPROVED, REJECTED, COMPLETED), RoleType (EMPLOYEE, APPROVER, ADMIN)
- Value Objects donde aplique (e.g. Email, RequestTitle con validación interna)
- Interfaces en domain/interfaces/: IRequestRepository, IUserRepository — solo firmas
- Patrón State: clase abstracta RequestState con transition(), clases concretas por estado

■ test_state.py — SUBMITTED→APPROVED válido; APPROVED→SUBMITTED lanza excepción

Patrones Creacionales

Los patrones deben estar en domain/ o application/factories/, nunca en la API ni en infra.

- Factory: RequestFactory.create(type, data) — retorna Request con estado inicial correcto

■ test_factory.py — RequestFactory.create("access", {...}) retorna objeto con status=SUBMITTED

- Builder: AuditLogBuilder con .set_event().set_actor().set_timestamp().build()

■ test_builder.py — builder sin campo obligatorio lanza excepción; completo retorna objeto válido

– Singleton: Settings o EventBusRegistry usando `__new__` o módulo-level

■ test_singleton.py — dos llamadas retornan el mismo objeto (id() idéntico)

Infraestructura

ORM en infrastructure/orm/, repositorios en infrastructure/repositories/. Nunca llamar ORM desde un use case.

– SQLAlchemy en infrastructure/database.py: create_engine, SessionLocal, Base

– Modelos ORM: UserModel, RequestModel, AuditLogModel, NotificationModel con ForeignKey y relationship

– SQLAlchemyRepository(IRequestRepository): save(), find_by_id(), find_by_role()

– SQLAlchemyRepository(IUserRepository): find_by_email(), save()

– Event bus en infrastructure/event_bus.py: publish(event), subscribe(event_type, listener)

■ test_repository.py — guardar Request y recuperar por ID retorna el mismo objeto

■ test_event_bus.py — suscribir listener y publicar evento llama al listener

API — Autenticación y Usuarios

JWT obligatorio en todos los endpoints protegidos. Middleware de rol reutilizable con dependency injection.

– POST /auth/login — recibe {email, password}, retorna {access_token, token_type, role}

– POST /auth/register — crea usuario con rol por defecto

– Dependency get_current_user en api/dependencies.py — decodifica JWT

– Dependency require_role("APPROVER") — lanza 403 si no coincide

– GET /users — solo ADMIN

■ test_auth.py — login correcto→token; login incorrecto→401; rol equivocado→403

Docker & DevOps

docker-compose.yml debe levantar todo con un solo comando. Sin variables hardcodeadas; usar .env.

- backend/Dockerfile: imagen base Python, pip install, comando uvicorn
- docker-compose.yml: servicios db (postgres:15), backend (depends_on: db), frontend
- Variables en .env: DATABASE_URL, SECRET_KEY, DOCKERHUB_USERNAME, etc.
- Verificar GitHub Actions despliega a Render dev tras merge a dev

■■■ *Depende de: Ayelén (entidades + interfaces + event bus — jueves). De ella dependen: Paula y toda la lógica de negocio de la demo.*

DTOs

Los DTOs son lo primero — definen el contrato API↔use case. Hacerlos antes de los use cases.

- CreateRequestDTO(BaseModel): campos obligatorios + validaciones Pydantic
- RequestResponseDTO: nunca exponer entidad de dominio directa al frontend
- ApproveRequestDTO, RejectRequestDTO: campo comment opcional
- NotificationDTO, AuditLogDTO

■ test_dtos.py — DTO con campo faltante lanza ValidationError; completo se instancia

Casos de Uso Principales

Cada use case recibe repositorio e interfaces por inyección de dependencias. Sin instanciar nada interno.

- CreateRequestUseCase: valida auth → RequestFactory.create() → repo.save() → event_bus.publish(REQUEST_CREATED) → retorna DTO
- ApproveRequestUseCase: valida rol APPROVER → State.transition() → publica REQUEST_APPROVED
- RejectRequestUseCase: igual que approve con REQUEST_REJECTED, requiere comentario
- GetRequestsByRoleUseCase: repo.find_by_role(role) → lista de RequestResponseDTO
- GetRequestDetailUseCase: repo.find_by_id(id) → detalle completo

■ test_create_request.py — mock repo y event_bus, verificar save() y publish() llamados

■ test_approve.py — EMPLOYEE intenta aprobar → excepción de permisos

■ test_get_by_role.py — mock retorna 3 requests → use case retorna lista de 3 DTOs

Patrón Observer

Listeners registrados en el event bus al iniciar la app, no dentro del use case.

- Interfaz IEventListener con método handle(event: DomainEvent)

- NotificationListener: crea Notification y guarda vía repositorio
- AuditListener: crea AuditLog usando AuditLogBuilder y lo guarda
- Registro en infrastructure/event_bus_setup.py o main.py al iniciar

■ test_observer.py — publicar REQUEST_SUBMITTED → NotificationListener.handle() llamado con datos correctos

Patrón Command

El Command encapsula la intención, el Handler la ejecuta. Sin lógica de negocio dentro del Command.

- Clase base Command (dataclass)
- ApproveRequestCommand: campos request_id, approver_id, comment
- RejectRequestCommand: campos request_id, approver_id, reason
- CommandHandler.handle(command) despacha al use case correcto

■ test_command.py — handle(ApproveRequestCommand(...)) invoca ApproveRequestUseCase.execute()

Patrón Strategy

La estrategia se selecciona según datos del request, no hardcodeada.

- Interfaz IApprovalStrategy con requires_additional_approval(request) → bool
- SimpleApprovalStrategy: siempre retorna False
- HighRiskApprovalStrategy: retorna True si criticidad/monto supera umbral
- ApprovalStrategySelector: recibe request y retorna estrategia correcta

■ test_strategy.py — request con risk=HIGH → selector retorna HighRiskApprovalStrategy

Patrón Template Method

El método base define el esqueleto del flujo; subclasses sobreescriben solo los pasos variables.

- Clase abstracta RequestProcessor con process(request): validate()→execute()→notify()→audit()
- Subclase concreta implementa los pasos que cambian (e.g. AccessRequestProcessor)

■ test_template_method.py — subclase ejecuta todos los pasos en orden (mocks o flags)

Auditoría

El AuditLog debe registrar quién, qué, cuándo y en qué estado quedó.

- Lógica centralizada en AuditListener (cubierto en Observer)
- Campos obligatorios: actor_id, action, entity_id, previous_state, new_state, timestamp

■ test_audit.py — aprobación genera AuditLog con previous_state=SUBMITTED, new_state=APPROVED

■■■ Depende de: Ayelén (auth endpoints) + Paula (POST /requests y GET /requests). Mientras espera: construir vistas con datos mock.

Login y Sesión

El token JWT debe guardarse en `st.session_state["token"]` y enviarse en cada request como `Authorization: Bearer`

- Vista `pages/login.py`: formulario email + password, botón Login
- Login exitoso: guardar token, role, `user_id` en `st.session_state` → redirigir a listado
- Login fallido: mensaje de error claro ("Credenciales incorrectas")
- Logout: botón en sidebar que limpia `st.session_state` y vuelve al login

■ `test_login_page.py` — 401→mensaje de error; 200→`st.session_state["token"]` tiene valor

Listado Principal

Llamar a `GET /requests` con el token. Si expiró, redirigir a login.

- Vista `pages/home.py`: tabla con columnas ID, Título, Estado, Fecha, Solicitante
- Estados con color: SUBMITTED (gris), APPROVED (verde), REJECTED (rojo), COMPLETED (azul)
- Cada fila clickeable redirige al detalle

■ `test_home_page.py` — mock retorna 3 solicitudes → 3 filas en tabla

Formulario de Creación

Validar campos obligatorios antes de llamar a la API. No enviar request vacío.

- Vista `pages/create_request.py`: campos según caso de negocio
- Campos vacíos muestran `st.warning()` antes de enviar
- Éxito: `st.success("Solicitud creada")` + redirección al listado

■ `test_create_form.py` — formulario vacío no llama a la API

Vista de Detalle

El estado actual debe mostrarse de forma prominente con badge de color.

– Vista `pages/detail.py`: recibe `request_id` por query param

– Secciones: datos generales, estado actual (badge), historial de estados, comentarios

■ `test_detail_page.py` — solicitud aprobada → estado muestra "APPROVED" en verde

■ Giuliana

Frontend: Acciones por Rol + Notificaciones + Historial + API Client

■■ Depende de: Pamela (páginas y sesión) + Paula (endpoints `/notifications`, `/audit`, `/approve`, `/reject`). Mientras espera: módulo de servicios con mocks.

Módulo de Servicios API

TODAS las llamadas HTTP pasan por `services/api_client.py`. Ninguna página llama a `requests.get()` directamente.

– Funciones: `get_requests()`, `get_request_detail()`, `create_request()`, `approve_request()`, `reject_request()`, `get_notifications()`, `get_audit_log()` — todas reciben token

– Errores centralizados: 401→limpiar sesión+login; 403→`st.error("Sin permisos")`; 500→`st.error("Error del servidor")`

■ `test_api_client.py` — mock con status 401 → función lanza excepción de sesión expirada

Acciones según Rol

Botones controlados por `st.session_state["role"]`. Autorización real en backend; UX no muestra acciones imposibles

– Botones visibles solo si `role == "APPROVER"` o `role == "ADMIN"` (coordinar con Pamela en `pages/detail.py`)

– Aprobar → confirmación (`st.dialog` o `st.expander`) → `approve_request()`

– Rechazar → campo razón obligatorio antes de confirmar → `reject_request()`

– Feedback: `st.success()` o `st.error()` + recarga de datos

■ `test_actions.py` — `role="EMPLOYEE"` → botones aprobar/rechazar no se renderizan

Vista de Notificaciones

Notificaciones cargadas con token del usuario. Mostrar mensaje si no hay ninguna.

– Vista `pages/notifications.py`: tipo, mensaje, timestamp

– Indicador en sidebar: número de notificaciones no leídas

– Lista vacía: "No tienes notificaciones"

■ test_notifications.py — mock vacío→mensaje vacío; mock 2 items→se muestran 2

Vista de Historial / Auditoría

Historial en orden cronológico, del más reciente al más antiguo.

– En pages/detail.py: sección "Historial de acciones" con actor, acción, estado anterior→nuevo, fecha

– Vista pages/audit.py para ADMIN: tabla general de todos los logs

■ test_audit_view.py — mock con 3 logs → se renderizan en orden correcto

■■■ Depende de: Ayelén (auth middleware) + Vaishali (use cases). De ella dependen: Pamela y Giuliana para conectar el viernes por la noche.

API — Endpoints de Solicitudes

Cada endpoint usa `get_current_user` de Ayelén y llama al use case de Vaishali. Sin lógica de negocio en el router.

- POST /requests — llama `CreateRequestUseCase` → 201 + `RequestResponseDTO`. Requiere token válido.
- GET /requests — llama `GetRequestsByRoleUseCase`, filtra por rol del token automáticamente
- GET /requests/{id} — llama `GetRequestDetailUseCase` → 200 o 404
- POST /requests/{id}/approve — requiere APPROVER/ADMIN, body {comment} opcional → 200 o 403
- POST /requests/{id}/reject — requiere APPROVER/ADMIN, body {reason} obligatorio

■ `test_requests_router.py` — sin token→401; EMPLOYEE→201; approve con EMPLOYEE→403

API — Notificaciones y Auditoría

Endpoints de solo lectura. No modifican estado.

- GET /notifications — notificaciones del usuario autenticado, ordenadas por fecha desc
- PATCH /notifications/{id}/read — marca como leída (opcional si da tiempo)
- GET /audit — solo ADMIN, retorna todos los logs
- GET /audit?request_id={id} — filtra por solicitud (para el detalle)

■ `test_notifications_router.py` — GET /notifications retorna solo las del usuario del token

UML Actualizado

Los diagramas deben reflejar el código real, no el diseño del PSet 4.

- Use Case Diagram /docs/uml/use_case.puml: actores (Employee, Approver, Admin) y casos de uso
- Class Diagram /docs/uml/class_diagram.puml: entidades + patrones con <>, <>
- Sequence Diagram /docs/uml/sequence_create.puml: POST /requests → use case → factory → repo → event_bus → listeners → response

- Exportar como PNG para el README

README Final

Alguien que nunca vio el proyecto debe poder correrlo leyendo solo el README.

- Descripción del problema + solución propuesta + alcance MVP
- Stack técnico con versiones
- Arquitectura hexagonal: diagrama de capas + qué va en cada capa
- Patrones aplicados: tabla Patrón / Dónde se usa / Por qué
- Cómo correr localmente: clonar, .env, pip install, uvicorn
- Cómo correr con Docker: docker compose up --build
- Ambientes dev/prod + links a Render
- Usuarios de prueba: tabla email / password / rol
- Flujo principal + referencia al sequence diagram
- Limitaciones conocidas

GitHub Project

- Verificar que todos los issues están en Done o con estado real
- Evidencia de al menos un PR por integrante con review

■ Mapa de Dependencias

Ayelén

■■■ domain + interfaces + ORM + auth

■■■ Vaishali

■ ■■■ use cases + patrones

■ ■■■ Paula

■ ■■■ routers (requests + notif + audit)

■ ■■■ Pamela (login · listado · formulario · detalle)

■ ■■■ Giuliana (acciones · notif · historial · api_client)

■■■ Paula (usa middleware de auth en sus routers)

Ayelén debe tener las interfaces de repositorio el **jueves** para que Vaishali trabaje con mocks sin esperar la implementación real.

Paula debe tener POST /requests y GET /requests el **viernes** para que Pamela y Giuliana conecten esa misma noche.